

Rintangan bagi Seekor Llama

Seekor Llama ingin menjelajahi Dataran Tinggi Andes. Ia memiliki peta dari dataran tinggi tersebut dalam bentuk papan berukuran $N \times M$ petak. Baris-barisnya diberi nomor dari 0 hingga $N - 1$ dari atas ke bawah, dan kolom-kolomnya diberi nomor dari 0 hingga $M - 1$ dari kiri ke kanan. Petak di baris i dan kolom j ($0 \leq i < N, 0 \leq j < M$) dinotasikan sebagai (i, j) .

Llama telah mempelajari iklim dari dataran tinggi tersebut dan menemukan bahwa seluruh petak dari suatu baris memiliki **temperatur** yang sama, dan seluruh petak dari suatu kolom memiliki **kelembapan** yang sama. Llama telah memberikan Anda dua buah *array* berisi bilangan bulat T dan H , dengan panjang masing-masing N dan M . Di sini, $T[i]$ ($0 \leq i < N$) menyatakan temperatur dari petak-petak di baris i , dan $H[j]$ ($0 \leq j < M$) menyatakan kelembapan dari petak-petak di kolom j .

Llama juga telah mempelajari flora dari dataran tinggi tersebut dan menyadari bahwa petak (i, j) **bebas dari vegetasi** jika dan hanya jika temperaturnya lebih dari kelembapannya, atau secara formal $T[i] > H[j]$.

Llama dapat bergerak di dataran tinggi tersebut hanya dengan melintasi **jalur yang valid**. Sebuah jalur yang valid adalah sebuah urutan petak-petak berbeda yang memenuhi kondisi-kondisi berikut:

- Setiap pasang petak yang bersebelahan dalam jalur memiliki sisi yang bersentuhan.
- Seluruh petak dalam jalur bebas dari vegetasi.

Tugas Anda adalah menjawab Q buah pertanyaan. Untuk setiap pertanyaan, Anda akan diberikan empat buah bilangan bulat: L, R, S , dan D . Anda harus menentukan apakah terdapat sebuah jalur yang valid sedemikian sehingga:

- Jalur dimulai dari petak $(0, S)$ dan berakhir di petak $(0, D)$.
- Seluruh petak dalam jalur berada di antara kolom L hingga R , inklusif.

Dijamin bahwa petak $(0, S)$ dan $(0, D)$ bebas dari vegetasi.

Detail Implementasi

Prosedur pertama yang harus Anda implementasikan adalah:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : sebuah *array* dengan panjang N yang menyatakan temperatur dari setiap baris.
- H : sebuah *array* dengan panjang M yang menyatakan kelembapan dari setiap kolom.
- Prosedur ini dipanggil tepat sekali untuk setiap kasus uji, sebelum seluruh pemanggilan `can_reach`.

Prosedur kedua yang harus Anda implementasikan adalah:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : bilangan-bilangan bulat yang mendeskripsikan sebuah pertanyaan.
- Prosedur ini dipanggil tepat Q kali untuk setiap kasus uji.

Prosedur ini harus mengembalikan nilai `true` jika dan hanya jika terdapat sebuah jalur yang valid dari petak $(0, S)$ ke petak $(0, D)$, sedemikian sehingga seluruh petak dalam jalur tersebut berada di antara kolom L hingga R , inklusif.

Batasan

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ untuk setiap i sehingga $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ untuk setiap j sehingga $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S, D \leq R$
- Petak $(0, S)$ dan $(0, D)$ bebas dari vegetasi.

Subsoal

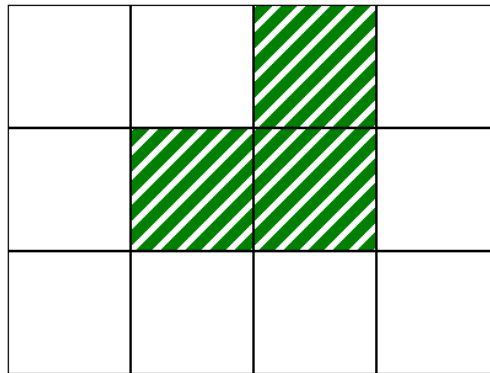
Subsoal	Nilai	Batasan Tambahan
1	10	$L = 0, R = M - 1$ untuk setiap pertanyaan. $N = 1$.
2	14	$L = 0, R = M - 1$ untuk setiap pertanyaan. $T[i - 1] \leq T[i]$ untuk setiap i sehingga $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ untuk setiap pertanyaan. $N = 3$ dan $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ untuk setiap pertanyaan. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ untuk setiap pertanyaan.
6	17	Tidak ada batasan tambahan.

Contoh

Perhatikan pemanggilan berikut:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

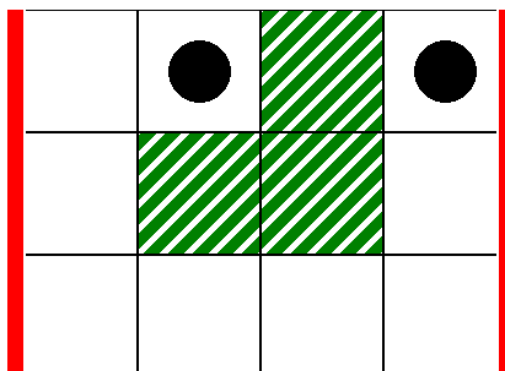
Ini diilustrasikan oleh peta berikut, di mana petak-petak putih bebas dari vegetasi:



Sebagai pertanyaan pertama, perhatikan pemanggilan berikut:

```
can_reach(0, 3, 1, 3)
```

Ini diilustrasikan oleh situasi pada peta berikut, di mana garis tebal vertikal mengindikasikan rentang kolom-kolom dari $L = 0$ hingga $R = 3$, dan lingkaran-lingkaran hitam menyatakan petak awal dan akhir:



Pada kasus ini, llama dapat bergerak dari petak (0,1) hingga petak (0,3) melintasi jalur yang valid sebagai berikut:

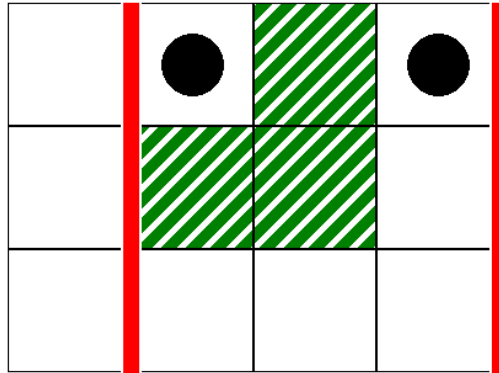
(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)

Sehingga, pemanggilan ini harus mengembalikan `true`.

Sebagai pertanyaan kedua, perhatikan pemanggilan berikut:

```
can_reach(1, 3, 1, 3)
```

Ini diilustrasikan oleh situasi pada peta berikut:



Pada kasus ini, tidak ada jalur yang valid dari petak (0,1) hingga petak (0,3), sedemikian sehingga seluruh petaknya berada di antara kolom 1 hingga 3, inklusif. Sehingga, pemanggilan ini harus mengembalikan `false`.

Contoh Grader

Format masukan:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Di sini, $L[k], R[k], S[k]$ dan $D[k]$ ($0 \leq k < Q$) merupakan parameter untuk pemanggilan `can_reach`.

Format keluaran:

```
A[0]
A[1]
...
A[Q-1]
```

Di sini, $A[k]$ ($0 \leq k < Q$) adalah 1 jika pemanggilan `can_reach(L[k], R[k], S[k], D[k])` mengembalikan `true`, dan 0 jika tidak.